

EXERCÍCIO PRÁTICO — AULA 3

Pipeline gráfico 3D com Python + Pygame

*Do modelo geométrico à imagem na tela***Disciplina: Computação Gráfica — Aula 3**

Material de apoio para desenvolvimento individual ou em grupo.

1. Objetivos

Ao concluir a atividade, o estudante deverá ser capaz de:

- Representar um objeto tridimensional por vértices e arestas.
- Aplicar rotações nos eixos X, Y e Z.
- Compreender a transformação de coordenadas do modelo para a câmera.
- Implementar projeção perspectiva e projeção ortográfica.
- Converter pontos 3D em coordenadas de tela e desenhar o resultado.
- Relacionar as funções do programa às etapas do pipeline gráfico.

2. Enunciado

Construa uma aplicação que represente um cubo tridimensional em uma janela 2D. O programa deverá definir os oito vértices e as doze arestas do cubo, aplicar transformações geométricas, projetar os pontos em coordenadas de tela e desenhar as arestas com Pygame.

A aplicação deverá permitir que o usuário altere a orientação do cubo, aproxime ou afaste a câmera e alterne entre projeção perspectiva e ortográfica. O objetivo não é utilizar um motor 3D pronto, mas tornar visíveis as etapas essenciais do pipeline gráfico.

3. Conceitos trabalhados

Etapa	Representação no programa
Modelagem	Lista de vértices e arestas do cubo
Transformação	Funções de rotação e composição de transformações
Câmera	Deslocamento pela variável <code>camera_z</code>
Projeção	Funções perspectiva e ortográfica
Rasterização	<code>pygame.draw.line</code>
Imagem final	Janela exibida ao usuário

4. Preparação do ambiente

Use Python 3.10 ou versão posterior. Recomenda-se criar um ambiente virtual para manter as dependências do exercício isoladas.

No Windows:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install pygame
```

No Linux ou macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pygame
```

Para confirmar a instalação:

```
python -c "import pygame; print(pygame.version.ver)"
```

Salve o código da seção seguinte como `aula3_pipeline.py` e execute com:

```
python aula3_pipeline.py
```

5. Código completo

O código abaixo implementa o modelo geométrico, as rotações, as duas projeções, a interação pelo teclado e a exibição das informações na tela.

```
import math
import pygame

# -----
# Configuracao da janela
# -----
WIDTH, HEIGHT = 800, 600
BACKGROUND = (18, 24, 38)
CUBE_COLOR = (80, 190, 255)
TEXT_COLOR = (235, 240, 248)

# Vertices do cubo no espaco 3D
VERTICES = [
    (-1, -1, -1), ( 1, -1, -1),
    ( 1,  1, -1), (-1,  1, -1),
    (-1, -1,  1), ( 1, -1,  1),
    ( 1,  1,  1), (-1,  1,  1),
]

# Cada tupla conecta dois vertices e forma uma aresta
EDGES = [
    (0, 1), (1, 2), (2, 3), (3, 0),
    (4, 5), (5, 6), (6, 7), (7, 4),
    (0, 4), (1, 5), (2, 6), (3, 7),
]

def rotate_x(point, angle):
    x, y, z = point
    c, s = math.cos(angle), math.sin(angle)
    return (x, y * c - z * s, y * s + z * c)
```

```

def rotate_y(point, angle):
    x, y, z = point
    c, s = math.cos(angle), math.sin(angle)
    return (x * c + z * s, y, -x * s + z * c)

def rotate_z(point, angle):
    x, y, z = point
    c, s = math.cos(angle), math.sin(angle)
    return (x * c - y * s, x * s + y * c, z)

def transform(point, angles):
    """Aplica rotacoes no modelo, na ordem X -> Y -> Z."""
    point = rotate_x(point, angles[0])
    point = rotate_y(point, angles[1])
    point = rotate_z(point, angles[2])
    return point

def project_perspective(point, focal_length, camera_z):
    """Projeta 3D em 2D; retorna None se o ponto estiver atras da camera."""
    x, y, z = point
    z_camera = z + camera_z
    if z_camera <= 0.1:
        return None
    scale = focal_length / z_camera
    screen_x = WIDTH / 2 + x * scale
    screen_y = HEIGHT / 2 - y * scale
    return (round(screen_x), round(screen_y))

def project_orthographic(point, scale):
    """Na ortografica, a profundidade nao altera o tamanho aparente."""
    x, y, _ = point
    screen_x = WIDTH / 2 + x * scale
    screen_y = HEIGHT / 2 - y * scale
    return (round(screen_x), round(screen_y))

def draw_cube(surface, projected, color):
    for start, end in EDGES:
        p1, p2 = projected[start], projected[end]
        if p1 is not None and p2 is not None:
            pygame.draw.line(surface, color, p1, p2, width=3)

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption('Pipeline grafico 3D - Aula 3')
    clock = pygame.time.Clock()
    font = pygame.font.SysFont('Arial', 20)

    angles = [0.0, 0.0, 0.0]
    camera_z = 6.0
    projection = 'perspectiva'
    running = True

    while running:
        dt = clock.tick(60) / 1000.0
        for event in pygame.event.get():

```

```

        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            elif event.key == pygame.K_p:
                projection = ('ortografica' if projection == 'perspectiva'
                              else 'perspectiva')

    keys = pygame.key.get_pressed()
    speed = 2.0 * dt
    if keys[pygame.K_a]: angles[1] -= speed
    if keys[pygame.K_d]: angles[1] += speed
    if keys[pygame.K_w]: angles[0] -= speed
    if keys[pygame.K_s]: angles[0] += speed
    if keys[pygame.K_q]: angles[2] -= speed
    if keys[pygame.K_e]: angles[2] += speed
    if keys[pygame.K_EQUALS] or keys[pygame.K_KP_PLUS]:
        camera_z = max(2.0, camera_z - 3.0 * dt)
    if keys[pygame.K_MINUS] or keys[pygame.K_KP_MINUS]:
        camera_z = min(12.0, camera_z + 3.0 * dt)

    transformed = [transform(v, angles) for v in VERTICES]
    if projection == 'perspectiva':
        projected = [project_perspective(v, 360, camera_z)
                     for v in transformed]
    else:
        projected = [project_orthographic(v, 130) for v in transformed]

    screen.fill(BACKGROUND)
    draw_cube(screen, projected, CUBE_COLOR)
    info = f'Projecao: {projection} | Camera Z: {camera_z:.1f}'
    screen.blit(font.render(info, True, TEXT_COLOR), (20, 18))
    screen.blit(font.render('A/D Y | W/S X | Q/E Z | +/- camera | P troca | ESC
sai',
                                True, TEXT_COLOR), (20, 48))

    pygame.display.flip()

    pygame.quit()

if __name__ == '__main__':
    main()

```

6. Controles da aplicação

Tecla(s)	Ação
A / D	Rotacionar no eixo Y
W / S	Rotacionar no eixo X
Q / E	Rotacionar no eixo Z
+ / -	Aproximar ou afastar a câmera
P	Alternar entre perspectiva e ortográfica
ESC	Encerrar a aplicação

7. Roteiro de desenvolvimento e análise

1. Execute o programa sem alterar o código e identifique o cubo, a câmera e a projeção.
2. Pressione as teclas de rotação separadamente. Observe qual coordenada dos vértices muda visualmente.
3. Altere a distância da câmera e registre o que acontece no modo perspectiva.
4. Pressione P e compare o comportamento com a projeção ortográfica.
5. Localize no código cada etapa do pipeline: modelagem, transformação, câmera, projeção e rasterização.
6. Modifique o valor da distância focal e o fator de escala ortográfico. Descreva o efeito visual.

8. Questões para discussão

7. Qual é a diferença entre um vértice no espaço 3D e um ponto na tela?
8. Em que momento o programa transforma coordenadas 3D em coordenadas 2D?
9. Por que a divisão pela profundidade z produz o efeito de perspectiva?
10. O que acontece quando um ponto se aproxima demais da câmera?
11. Por que a projeção ortográfica não altera o tamanho aparente conforme a profundidade?
12. Quais etapas de um pipeline gráfico real foram simplificadas neste exercício?
13. Por que o Pygame pode ser utilizado para estudar conceitos 3D mesmo desenhando primitivas 2D?

9. Extensões opcionais

- Preencher as faces do cubo com `pygame.draw.polygon`.
- Implementar remoção de faces ocultas usando normais ou ordenação por profundidade.
- Adicionar uma pirâmide ou um segundo cubo à cena.
- Exibir os eixos coordenados X, Y e Z e uma grade no plano XZ.
- Adicionar translação nos eixos X, Y e Z.
- Controlar a orientação com o mouse.

10. Produto esperado

A entrega deverá conter:

- O programa Python executável.
- Uma captura de tela da aplicação funcionando.
- Uma breve explicação das funções de transformação e projeção.
- Uma comparação entre os modos perspectiva e ortográfica.
- As respostas às questões de discussão.

11. Síntese

A atividade transforma o pipeline gráfico em uma sequência observável: um modelo geométrico é descrito por dados, transformado por operações matemáticas, posicionado em relação à câmera, projetado no plano da imagem e finalmente desenhado na tela. A comparação entre perspectiva e ortográfica permite perceber como diferentes modelos de projeção produzem imagens distintas a partir da mesma cena 3D.